

stream_cmd

[TOC]

项目简介

stream_cmd文件夹下的sample提供了基础出图和通信的示例。

该示例展示了以下功能：

- 获取数据
- 显示出图
- 发送命令

该示例从上层（应用层）到下层（通信协议层）分别依赖了：

基本功能组件（commom文件夹）中的data ‘v4l2_camera ‘drm_display ‘config模块

特殊功能组件（components文件夹）中的cmd模块

接口适配层（interfaces文件夹）中的libircam ‘libircmd

驱动层（drivers文件夹）中的libiruart ‘libirv4l2。

程序编译方式

linux平台

stream_cmd文件夹下提供了CMakeList.txt文件，支持32位或64位sample程序的编译，相应的编译命令如下：

编译32位的sample

```
cmake -DBUILD32=ON
make
```

编译64位的sample

```
cmake -DBUILD32=OFF
make
```

注意：在sample编译之前，需要事先获取与sample相同位数（32位或64位）的libircam.so ‘libircmd.so ‘libiruart.so三个动态库，并将其放于drivers文件夹下，否则直接使用CMakeList.txt编译sample时会提示错误。

程序运行方式

./sample

如在RV1126平台上运行：./sample config/stream_RV1126.conf

修改内核驱动

本工程是基于RV1126的显示出图，需要修改内核以支持MIPI和UART。

相关文件都在RV1126_configure_files文件夹下，也提供了编译好的zboot.img

打开串口配置

内核文件 kernel/drivers/usb/serial/Makefile 把原来里面的配置项直接改为y,表示编译进内核，需要添加使能下面几款芯片的驱动

```
obj-y += ch341.o
obj-y += cp210x.o
obj-y += ftdi_sio.o
obj-y += p12303.o
```

举例：

```
obj-$(CONFIG_USB_SERIAL_CH341) += ch341.o
#修改为
obj-y += ch341.o
```

修改配置和设备树

配置文件aio-rv1126-opemwrt.mk 放置在内核目录device/rockchip/rv1126_rv1109下。新设备树文件rv1126-firefly-openwrt.dts

放置在内核目录kernel/arch/arm/boot/dts下。

替换USB驱动

需要把驱动文件uvc_v4l2.c.c 替换driver/media/usb

添加MIPI驱动

需要把驱动文件rs300-mipi.c 放到driver/media/i2c下

在内核driver/media/i2c/Makefile里面添加自定义的驱动

obj-y += rs300.o

需要注意，驱动的宽高需要添加机芯中对应的宽高（line313）

```
static struct rs300_framesize rs300_framesizes[] = {
    { /* 640*/
        .width      = 640,
        .height     = 900,
        .max_fps = {
            .numerator = 30,
            .denominator = 1,
        },
        .code = MEDIA_BUS_FMT_YUYV8_2X8,
    },
    { /* 1280 */
        .width      = 1280,
        .height     = 1797,
        .max_fps = {
            .numerator = 30,
            .denominator = 1,
        },
        .code = MEDIA_BUS_FMT_YUYV8_2X8,
    }
};
```

修改MIPI出图datatype

rv1126当前驱动，按照如下修改以支持datatype 0x2A。mipi通道0：kernel/drivers/media/platform/rockchip/cif/capture.c 修改get_data_type函数，将0x1e改为0x2a mipi通道1：kernel/drivers/media/platform/rockchip/isp/regs.h 398行里面改成如下：

```
#define CIF_CSI2_DT_YUV422_8b          0x2A
```

编译内核

需要参考RV1126内核的[官网编译方式](#)，获取源码并编译。

在内核目录下调用build.sh文件如下操作：

```
./build.sh device/rockchip/rv1126_rv1109/aio-rv1126-opemwrt.mk #进行切换配置 ./build.sh kernel #编译内核,生成新内核文件kernel/zboot.img
```

编译的内核为在kernel目录下的zboot.img

更新内核

```
adb push ./zboot.img / #推送内核
adb shell #进入shell
dd if=zboot.img of=/dev/mmcblk0p3 #更新内核，直接内核文件覆盖分区3即可
reboot #重启系统
```

MIPI出图节点

video5:MIPI0 virtual1

video6:MIPI0 virtual2

video7:MIPI0 virtual3

video8:MIPI0 virtual4

video11:MIPI1节点，需要先开启video11，才能使用下面的VC（另开一个shell，输入v4l2-ctl -d /dev/video11 --stream-mmap）

video12:MIPI1 virtual1

video13:MIPI1 virtual2

video14:MIPI1 virtual3

出图

出图功能使用的是V4L2框架，调用mipi驱动以获取数据

在sample.cpp的main函数中，以下代码打开设备用于获取出图的handle，设备号在config目录下stream.conf中进行配置

```
printf("init stream handle\n");
stream_frame_info.image_name = (char)product_config.camera.v4l2_config.image_stream.device_name.data();
ir_video_handle_create(&ir_image_video_handle);
stream_frame_info.image_driver_handle = (Irv4l2VideoHandle_t*)irv4l2_handle_create(ir_image_video_handle);
```

在sample.cpp的main函数中，以下代码会寻找uart设备，打开并获取命令的handle。

```
printf("init control handle\n");
IrControlHandle_t* ir_control_handle = NULL;
ir_control_handle_create(&ir_control_handle);
IruartHandle_t* iruart_handle = NULL;
iruart_handle = iruart_handle_create(ir_control_handle);

FILE* fp;
char buffer[80] = { 0 };
char dev_node[20] = "/dev/";
fp = popen("ls /dev | grep \"ttyUSB*\" ", "r");
fgets(buffer, sizeof(buffer), fp);
printf("content of ls /dev | grep \"ttyUSB*\" is %s\n", buffer);
strcat(dev_node, buffer);
printf("dev_node is %s\n", dev_node);

int i;
for (i = sizeof(dev_node) - 1; i < sizeof(dev_node); i--)
{
    if (dev_node[i] == '\\n')
    {
        dev_node[i] = 0;
        break;
    }
}
ret = ir_control_handle->ir_control_open(iruart_handle, dev_node);
printf("open control node:%d\n", ret);
stream_frame_info.ircmd_handle = ircmd_create_handle(ir_control_handle);
ret = basic_video_stream_continue(stream_frame_info.ircmd_handle);
printf("basic_video_stream_continue:%d\n", ret);    //control mipi start
```

特别注意：

1. 对于uart串口波特率，未设置将默认使用115200波特率与机芯交互，如需设置其他波特率，见cmd.cpp文件内search_com函数内ir_control_init处调用

在stream.conf中，配置需要的出图模式，机芯会根据对应的模式设置相应分辨率，可选{ “yuyv_image”，“nv12_image”，“nv12_and_temp”，“yuyv_and_temp”，“uyvy_image” }

```
"camera":{
    ...
    "format":"nv12_and_temp",
    ...
}
```

本示例是根据解析config目录中的stream.conf配置初始化参数调用，如果需要调整分辨率，可以按照该目录中stream_config.md的说明进行调整

- 选择对应的video节点
- 在camera段中修改数据源的宽高fps

```
"v4l2_stream":[
{
    "device_name":"/dev/video38",
    "type":"image",
    "fps":30,
    "dev_width":640,
    "dev_height":387
},
{
    "device_name":"/dev/video40",
    "type":"temp",
    "fps":30,
    "dev_width":640,
    "dev_height":514
}
]
```

- sample.cpp的load_stream_frame_info：需要输入对应的宽高，以及设置对应的复合格式。

数据源640X900对应：640x512 nv12出图 + 640x2行信息行 + 640x512 y16温度信息 + 640x2 dummy行

数据源1280X1797对应：1280x1024 nv12出图 + 1280x2行信息行 + 280x1024 y16温度信息 + 1280x3 dummy行

发送命令

当运行sample之后，main函数会新建一个cmd_function线程，线程中包含了几个简单的调用命令示例，见cmd.cpp，根据输入的数字来发送命令。

```
//command thread function
void* cmd_function(void* threadarg)
{
    int isRUNNING = 1;
    int cmd;
    while (isRUNNING)
    {
        printf("please select cmd type\n");
        printf("1:get_device_info 2:basic_ffc_update 3:palette_operate 4:temp_measure 5:data_update\n");
        printf("6:set_image_scene_mode 7:image_effecttion\n");
        scanf("%d", &cmd);
        command_sel(cmd, (StreamFrameInfo_t*)threadarg);
    }
    printf("cmd thread exit!!\n");
    return NULL;
}
```